



Dokumentation

get_playing_field.py

Für das Spielfeld gab es zwei Möglichkeiten:

1. Möglichkeit: Liste mit der Länge 25
2. Möglichkeit 5 Listen der Länge 5 in einer Liste

Entschieden wurde sich für die 2te Möglichkeit, da man einzelne Zellen des Spielfelds mit y-Koordinate (Index der Gesamtliste) und x-Koordinate (Index der einzelnen Listen) bestimmen kann. Dazu das Modul `get_playing_field.py`:

```
"""Das Modul erstellt das 5x5 Spielfeld."""

import random
from typing import List

def get_playing_field(bomb_count: int) → List[List[int]]:
    """Generiert ein 5x5 Spielfeld mit eingetragenen Nummern"""
    playing_field_list = []
    binary_playing_field = []

    for i in range(bomb_count):
        playing_field_list.append(0)
    for i in range(25 - bomb_count):
        playing_field_list.append(1)

    random.shuffle(playing_field_list)
    for i in range(5):
        binary_playing_field.append(playing_field_list[i * 5 : i * 5 + 5])

    directions = [(0, 1), (1, 1), (1, 0), (1, -1), (0, -1), (-1, -1), (-1, 0), (-1, 1)]
    playing_field = []
    current_row = 0
    current_column = 0

    for row in binary_playing_field:
        result_row = []
        for cell in row:
            bomb_count = 0
            for direction in directions:
                search_column = current_column + direction[0]
```

```

        search_row = current_row + direction[1]
        if search_row >= 5 or search_row < 0:
            continue
        if search_column >= 5 or search_column < 0:
            continue
        if binary_playing_field[search_row][search_column] == 0:
            bomb_count += 1

    if cell == 0:
        result_row.append(9)
    else:
        result_row.append(bomb_count)

    current_column += 1
    playing_field.append(result_row)
    current_row += 1
    current_column = 0
    return playing_field

if __name__ == "__main__":
    print(get_playing_field(6))

```

Dazu wurde die Funktion `get_playing_field.py` geschrieben. Ursprünglich war die Aufgabe der Funktion nur die Eine Liste mit 5 Listen aus Nullen und Einsen zu returnen. Dabei wird in eine Liste 5 mal die 0 hinzugefügt und 20 mal die 1. Diese wird dann mit der `random` Bibliothek zufällig gemischt.

```
[[1, 1, 1, 0, 1], [1, 0, 1, 1, 1], [1, 1, 1, 1, 1], [1, 0, 1, 1, 1], [1, 0, 1, 1, 0]]
```

Dabei waren 0er die Bomben und 1er die freien Felder.

Später wurde die Funktion mit dem `bomb_count` Parameter erweitert, sodass auch komplexere Spielfelder generiert werden können mit mehr Bomben. Außerdem wurden den freien Zellen Zahlen übergeben der Anzahl der benachbarten Bomben. Dabei werden in einer for Schleife für jede freie Zelle die 8 benachbarten Felder beobachtet, ob eine Bombe vorhanden ist. Die Anzahl der Bomben wird der Zelle als Integer übergeben. Die Bomben werden mit 9 markiert, da freie Zellen maximal 8 benachbarte Bomben haben können.

```
if __name__ == "__main__":
    print(get_playing_field(6))
```

```
[[9, 2, 9, 2, 9], [1, 2, 1, 2, 1], [0, 1, 2, 2, 1], [1, 2, 9, 9, 1], [1, 9, 3, 2, 1]]
```

Das funktioniert so, dass man die Liste als Matrix mit Koordinaten bestimmt. Dabei ist die y-Koordinate immer der Index in der Gesamtliste und die x-Koordinate der Index in den einzelnen

Listen. Die Directions geben dann mit den Koordinaten der jetzigen Zelle die Koordinaten der benachbarten Zellen wieder, die nach Bomben überprüft werden.

print_field_as_table.py

Eine weitere wichtige Funktion ist die `print_field_as_table()` in dem `print_field_as_table.py` Modul:

```
"""Printet das Spielfeld"""

from typing import List

def print_field_as_table(field: List[List[str]]) → None:
    """Nimmt Spielfeld als Liste als Parameter und printet Spielfeld"""
    print(" A B C D E\n")

    for i, row in enumerate(field):
        if isinstance(row[0], int):
            print_row = map(str, row)
        else:
            print_row = row
        print(str(i + 1) + " " + " ".join(print_row))
    print("")
```

Diese nimmt die Liste an Listen (Matrix) als Parametern und gibt sie in einem schönen Format für das Terminal aus:

```
field = [
    ["x", "x", "x", "x", "x"],
    ["x", "x", "x", "x", "x"],
    ["x", "x", "x", "x", "x"],
    ["x", "x", "x", "x", "x"],
    ["x", "x", "x", "x", "x"],
]

if __name__ == "__main__":
    print_field_as_table(field)
```

```
A B C D E
```

```
1 x x x x x
2 x x x x x
3 x x x x x
4 x x x x x
5 x x x x x
```

Dabei werden die einzelnen Zeilen des Feldes in Listen aus Strings gemacht und mit Leerzeichen gejoinet. Außerdem werden vor den Zeilen die Indexe der Zeilen ausgegeben ($i + 1$). Über dem Feld

wird auch noch für die Spaltenkoordinaten A, B, C, D, E angezeigt, um die Koordinatenangabe beim Scannen zu erleichtern.

get_end_screen_field.py

Ein weiteres Modul des Programms ist get_end_screen_field.py:

```
"""Kreiert das fertige Feld mit "x" als Bomben"""

from typing import List

def get_end_screen_field(numbers_field: List[List[int]]) → List[List[str]]:
    """Macht das Nummer Feld schön für die Losing Page"""
    end_screen_field = []
    for row in numbers_field:
        result_row = []
        for c in row:
            if c == 9:
                result_row.append("x")
            else:
                result_row.append(str(c))
        end_screen_field.append(result_row)
    return end_screen_field
```

Die Aufgabe dieser Funktion ist es das Feld anzuzeigen, wenn der Spieler verliert. Da das ursprüngliche playing_field, das aus Integer besteht und 9 für eine Bombe steht, nicht schön formatiert ist, um im Terminal auszugeben, werden die 9er für "x" ausgetauscht und die restlichen freien Zellen werden zum String umgewandelt. Damit lässt sich das Feld schön ausgeben so als hätte es jemand fertig gespielt.

win_condition.py

Die win_condition() bestimmt, ob das Spiel gewonnen wurde oder nicht. Der Gedanke hinter der Funktion ist, dass man alle Zellen aufdecken muss, die keine Bomben sind. Dabei braucht die Funktion auch wieder als Parameter das jetzige Spielfeld und den bomb_count:

```
"""Funktion, die das Gewinnen vom Spiel bestimmt"""

from typing import List

def win_condition(shown_field: List[List[str]], bomb_count: int) → bool:
    """Wenn nur noch 5 unaufgedeckte Felder → Spiel gewonnen"""
    x_count = 0
    for row in shown_field:
        for cell in row:
            if cell == "x":
```

```
x_count = x_count + 1
return x_count <= bomb_count
```

Die Funktion zählt alle unaufgedeckten Zellen. Wenn diese der bomb_count übereinstimmen oder kleiner sind zählt das Spiel als gewonnen.

Main.py

```
"""Main Module Des Spiels"""

import sys
from get_playing_field import get_playing_field
from print_field_as_table import print_field_as_table
from win_condition import win_condition
from get_end_screen_field import get_end_screen_field

def main() → None:
    """main funktion des Spiels"""

    allowed_characters = {
        "a": 0,
        "b": 1,
        "c": 2,
        "d": 3,
        "e": 4,
    }
    allowed_numbers = {1: 0, 2: 1, 3: 2, 4: 3, 5: 4}
    shown_field = [
        ["x", "x", "x", "x", "x"],
        ["x", "x", "x", "x", "x"],
        ["x", "x", "x", "x", "x"],
        ["x", "x", "x", "x", "x"],
        ["x", "x", "x", "x", "x"],
    ]
    error_message = ""
    print("Willkommen in SweeperMine. Deine Aufgabe ist es Felder aufzudecken.")
    print("Decke Felder auf indem du die Koordinaten angibst: (A1, C5, ...)\n")

    while True:
        bomb_count = 6
        difficulty = input(
            "Welche Schwierigkeit wählen Sie? (e: einfach, n: normal, s: schwer)\n"
        ).lower()

        match difficulty:
```

```

case "e":
    bomb_count = 4
    difficulty = "Einfach"
case "n":
    bomb_count = 6
    difficulty = "Normal"
case "s":
    bomb_count = 10
    difficulty = "Schwer"
case _:
    difficulty = "Normal"

playing_field = get_playing_field(bomb_count)

while not win_condition(shown_field, bomb_count):
    print("\n" * 15)

    if difficulty:
        print("Schwierigkeit: " + difficulty + "\n")
        difficulty = ""

    if error_message:
        print(error_message + "\n")
        error_message = ""

    print_field_as_table(shown_field)

    chosen_cell = input().lower()
    row_numbers = [int(c) for c in list(chosen_cell) if c.isdigit()]
    column_chars = [c for c in list(chosen_cell) if not c.isdigit()]

    if column_chars:
        if "q" in column_chars:
            break

    if not row_numbers or not column_chars:
        error_message = "Bitte eine Koordinate eingeben. z.B: (a1, c2, e4, d1)"
        continue

    try:
        chosen_row = allowed_numbers[row_numbers[0]]
        chosen_column = allowed_characters[column_chars[0]]

        if playing_field[chosen_row][chosen_column] == 9:
            print("\n" * 15)
            print("Verloren!\n")
            print_field_as_table(get_end_screen_field(playing_field))

```

```

        shown_field = [
            ["x", "x", "x", "x", "x"],
            ["x", "x", "x", "x", "x"],
            ["x", "x", "x", "x", "x"],
            ["x", "x", "x", "x", "x"],
            ["x", "x", "x", "x", "x"],
        ]
        break

    shown_field[chosen_row][chosen_column] = str(
        playing_field[chosen_row][chosen_column]
    )

except:
    error_message = "Ungültige Eingabe"

if win_condition(shown_field, bomb_count):
    print("\n" * 15)
    print_field_as_table(shown_field)
    print("Gewonnen!\n")
    shown_field = [
        ["x", "x", "x", "x", "x"],
        ["x", "x", "x", "x", "x"],
        ["x", "x", "x", "x", "x"],
        ["x", "x", "x", "x", "x"],
        ["x", "x", "x", "x", "x"],
    ]

end_screen_input = input("Nochmal spielen? (y/n)\n").lower()

if end_screen_input == "y":
    continue

if end_screen_input in ("n", "q"):
    sys.exit(1)

if __name__ == "__main__":
    main()

```

Am Anfang werden alle Module importiert.

Dann werden zuerst ein paar wichtige Variablen definiert:

```

allowed_characters = {
    "a": 0,
    "b": 1,
    "c": 2,
    "d": 3,

```

```

    "e": 4,
}
allowed_numbers = {1: 0, 2: 1, 3: 2, 4: 3, 5: 4}
shown_field = [
    ["x", "x", "x", "x", "x"],
    ["x", "x", "x", "x", "x"],
    ["x", "x", "x", "x", "x"],
    ["x", "x", "x", "x", "x"],
    ["x", "x", "x", "x", "x"],
]
error_message = ""

```

Die `allowed_characters` legen fest, welche Buchstaben-Eingaben welchen Index in den Listen des Spielfelds bestimmen. Zum Beispiel steht der Buchstabe A immer für die erste Spalte, also den Index 0 in den inneren Listen des Spielfelds.

Die `allowed_numbers` bestimmen, welche Zahlen-Eingaben des Spielers welchen Zeilen-Index im Spielfeld repräsentieren. Beispielsweise steht die Zahl 1 für die erste Zeile.

Das `shown_field` ist das sichtbare Spielfeld, das der Spieler im Terminal angezeigt bekommt und absuchen kann. Dieses Feld entscheidet letztlich auch darüber, ob die `win_condition` erfüllt ist oder nicht.

Die `error_message` ist zunächst ein leerer String und bekommt dann einen Wert, falls der Spieler ungültige oder fehlerhafte Koordinaten eingibt. Diese Nachricht wird dem Spieler entsprechend angezeigt.

Danach startet die eigentliche `while`-Schleife des Spiels.

Zuerst wird der `bomb_count` standardmäßig auf 6 gesetzt. Anschließend wird der Spieler gefragt, welche Schwierigkeit er spielen möchte. Je nach Eingabe wird der `bomb_count` mithilfe eines `switch case` angepasst.

Anschließend wird das Spielfeld zufällig generiert und die erste Spielrunde beginnt. Diese läuft ebenfalls in einer `while`-Schleife, solange die `win_condition` nicht erfüllt ist. Das heißt: Die Schleife endet, sobald der Spieler gewonnen hat.

Bevor der Spieler scannen darf, werden (falls vorhanden) die `error_message` und die gewählte Schwierigkeit ausgegeben. Danach wird das aktuelle `shown_field` angezeigt und der Spieler kann durch Eingabe von Koordinaten (z.B. "a1") ein Feld scannen. Aus dieser Eingabe werden die jeweiligen Indexe für das Spielfeld bestimmt.

Anschließend wird in einem `try-except`-Block geprüft, ob die Eingabe gültig ist. Falls nicht, wird die `error_message` auf "Ungültige Eingabe" gesetzt und der Spieler muss es erneut versuchen. Bei gültiger Eingabe wird geprüft, ob sich an der gewählten Stelle im `playing_field` eine Bombe befindet. Falls ja, endet das Spiel sofort. Falls keine Bombe vorhanden ist, wird das `shown_field` aktualisiert und das Feld erhält den Wert aus dem `playing_field`.

Wenn der Spieler gewonnen hat, wird die Schleife beendet und es erscheint die Nachricht "Gewonnen". Danach wird das Spielfeld zurückgesetzt und der Spieler wird gefragt, ob er noch eine Runde spielen möchte.

Falls der Spieler eine Bombe getroffen hat, wird ebenfalls die Schleife verlassen. In diesem Fall wird noch das komplette gelöste Spielfeld über die Funktion `get_end_screen_field()` angezeigt. Danach folgt wieder die Abfrage, ob der Spieler eine neue Runde starten möchte.

Ein neues Spiel wird durch Eingabe von "y" gestartet. Bei Eingabe von "n" oder "q" wird das Programm beendet.

Zusätzlich kann der Spieler jederzeit das laufende Spiel oder das gesamte Programm durch Eingabe von "q" abbrechen.

Versionen der genutzten Bibliotheken

```
coverage==7.6.12
mypy==1.15.0
mypy-extensions==1.0.0
pylint==3.3.4
```

Programm ausführung

1. Zuerst die Zip entpacken
3. main.py ausführen

```
python source/main.py
```

Nutzerinterface

Start-Interface

Willkommen in SweeperMine. Deine Aufgabe ist es Felder aufzudecken.
Decke Felder auf indem du die Koordinaten angibst: (A1, C5, ...)

Welche Schwierigkeit wählen Sie? (e: einfach, n: normal, s: schwer)

Hier kann man mit "e", "n", "s" die Schwierigkeit des Spiels wählen. Einfach eines der drei Optionen ins Terminal eingeben.

Schwierigkeit: Normal

A B C D E

```
1 x x x x x
2 x x x x x
3 x x x x x
4 x x x x x
5 x x x x x
```

Das Spiel beginnt und der Spieler darf scannen. Zum Beispiel ("a1")

A B C D E

```
1 0 x x x x
2 x x x x x
3 x x x x x
4 x x x x x
5 x x x x x
```

Gewonnen:

A B C D E

```
1 0 0 1 1 1
2 1 1 1 x 1
3 x 2 3 3 2
4 2 3 x x 2
5 x 2 3 x 2
```

Gewonnen!

Nochmal spielen? (y/n)

Mit "y" oder "n" nochmal spielen

Verloren:

Verloren!

A B C D E

```
1 x 3 2 x 1
2 x 4 x 2 1
3 4 x 4 2 0
4 x x x 3 2
5 2 3 3 x x
```

Nochmal spielen? (y/n)

Möglichkeit nochmal zu Spielen wieder gegeben.

Metriken

Pylint

.pylintrc

```
[MASTER]
init-hook='import sys; sys.path.append("./source")'
[MESSAGES CONTROL]
```

```
disable=  
    W0702, # No exception type(s) specified (bare-except)  
    R0915 # Too many statements in a function/method (too-many-statements)  
  
[FORMAT]  
max-statements=60
```

get_playing_field.py

```
.venv(base) lucius.lechner@MAC-L69H394NCW source % pylint get_playing_field.py  
  
-----  
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)
```

print_field_as_table.py

```
.venv(base) lucius.lechner@MAC-L69H394NCW source % pylint print_field_as_table.py  
  
-----  
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)
```

get_end_screen_field.py

```
.venv(base) lucius.lechner@MAC-L69H394NCW source % pylint get_end_screen_field.py  
  
-----  
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)
```

win_condition.py

```
.venv(base) lucius.lechner@MAC-L69H394NCW source % pylint win_condition.py  
  
-----  
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)
```

main.py

```
.venv(base) lucius.lechner@MAC-L69H394NCW source % pylint main.py  
  
-----  
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)
```

Mypy

```
.venv(base) lucius.lechner@MAC-L69H394NCW source % mypy get_playing_field.py
Success: no issues found in 1 source file
.venv(base) lucius.lechner@MAC-L69H394NCW source % mypy print_field_as_table.py
Success: no issues found in 1 source file
.venv(base) lucius.lechner@MAC-L69H394NCW source % mypy get_end_screen_field.py
Success: no issues found in 1 source file
.venv(base) lucius.lechner@MAC-L69H394NCW source % mypy win_condition.py
Success: no issues found in 1 source file
.venv(base) lucius.lechner@MAC-L69H394NCW source % mypy main.py
Success: no issues found in 1 source file
```

Coverage

```
.venv(base) lucius.lechner@MAC-L69H394NCW PythonProgramm % coverage report
Name                               Stmts Miss Cover
-----
source/__init__.py                  0     0 100%
source/get_end_screen_field.py      11     0 100%
source/get_playing_field.py         39     1 97%
source/main.py                     67    26 61%
source/print_field_as_table.py       9     0 100%
source/win_condition.py              8     0 100%
tests/__init__.py                   0     0 100%
tests/test_get_end_screen_field.py  13     1 92%
tests/test_get_playing_field.py     19     1 95%
tests/test_main.py                  25     4 84%
tests/test_print_field_as_table.py  31     1 97%
tests/test_win_condition.py         14     1 93%
-----
TOTAL                               236    35 85%
```

Unittest

```
.venv(base) lucius.lechner@MAC-L69H394NCW PythonProgramm % python -m unittest discover
.....Willkommen in SweeperMine. Deine Aufgabe ist es Felder aufzudecken.
Decke Felder auf indem du die Koordinaten angibst: (A1, C5, ...)
```

Schwierigkeit: Normal

A B C D E

```
1 x x x x x
2 x x x x x
3 x x x x x
4 x x x x x
5 x x x x x
```

.Willkommen in SweeperMine. Deine Aufgabe ist es Felder aufzudecken.
Decke Felder auf indem du die Koordinaten angibst: (A1, C5, ...)

Gewonnen!

.....

Ran 13 tests in 0.001s

OK

.venv(base) lucius.lechner@MAC-L69H394NCW PythonProgramm %

Selbsteinschätzung liegt im Root-Directory :D



Lucius Lechner